

Overview & System Architecture

?

What is ESP-NOW

The robot is controlled by handheld ESP32 controllers over ESP-NOW, a connectionless, low-latency wireless protocol built into the ESP32 that sits below WiFi's usual AP/station overhead. There's no pairing handshake, no IP stack, and no dropped sessions to manage.

!

Why ESP-NOW (not WiFi/UDP or Bluetooth)

- ▶ ~1ms typical latency vs. tens of ms for a WiFi socket
- ▶ No access point required — works in the field or at a hackerspace
- ▶ Peer-to-peer: any ESP32 can talk to any other it knows the MAC of
- ▶ Packets map directly onto C structs — no JSON, no serialization

One Robot, Multiple Controllers

There is one robot, driven by multiple different controllers — it has no idea which one it's hearing from at any moment. The robot runs one fixed firmware image no matter which controller is talking to it; it just listens for the same packet types, from any peer, on the same MAC/channel.

Device	Role	MAC Address
Motor receiver (robot)	Drives motors, relay/lights	C0:CD:D6:CA:0E:4C
Relay receiver (robot)	Switches lights/accessories	D4:E9:F4:E6:A7:B8
Single-joystick controller	Sender — drive + lights	controller-specific
Dual-joystick controller	Sender + receiver — telemetry	D4:E9:F4:E6:07:F8

Because the protocol is just two small, documented C structs sent as raw ESP-NOW packets, any bare ESP32 — flashed with nothing more than the struct definitions and the robot's MAC address — can control the robot. No authentication, no proprietary handshake.

The Robot



Fixed Firmware, Two Receiver Boards

All devices are locked to a fixed WiFi channel (channel 1) so they can hear each other without joining a network. Peers are registered explicitly via `esp_now_add_peer()` using each other's hardcoded MAC address — clean unicast, no broadcast noise.

- ▶ Motor receiver — drives the motors from continuous drive packets
- ▶ Relay receiver — switches lights/accessories from one-shot events

Telling Packets Apart — By Length

Each function gets its own small, fixed-size, packed C struct. The receiver tells packet types apart by length — a zero-overhead form of message typing that needs no header byte.

```
typedef struct __attribute__((packed)) {
    uint16_t x;    // joystick X, 0-4095
    uint16_t y;    // joystick Y, 0-4095
    uint32_t seq;  // rolling sequence #
    uint32_t ms;   // sender timestamp
} JoyPacket;    // 12 bytes – drive
```

```
typedef struct __attribute__((packed)) {
    uint8_t state; // 0 = off, 1 = on
    uint32_t seq;
} LightPacket;    // 5 bytes – accessory event
```



Talking Back: the Heartbeat

The motor receiver sends a periodic heartbeat packet back to any listening controller, roughly every 500ms — RSSI, relay state, uptime. A screen-equipped controller renders this as a live HUD; a simpler controller just never asks for it. The robot doesn't need to know or care which is connected.

Reliability, Handled at the Application Layer

- ▶ Continuous streams (drive) are self-healing — a dropped packet is superseded 5ms later; a receiver-side timeout is a fail-safe stop
- ▶ One-shot events (light toggles) are made reliable by resending a few times in a burst rather than waiting on an ACK

Controller 1 — Single Joystick (simple)

1

ANALOG STICK

1

CLICK BUTTON

0

DISPLAYS

This is the minimal controller — one analog stick, one click button, no display. It only ever sends; it never listens for anything back from the robot.

Behavior

Joystick X/Y → packed into a JoyPacket and sent to the robot continuously (~every 5ms)

Single click cycles four drive speed presets (25% / 50% / 75% / 100%)

Double click (2nd click within ~350ms) fires a one-shot LightPacket toggle

Why It's Built This Way

One-way, fire-and-forget — the robot never talks back to this controller. One physical button drives two logically distinct systems (speed and lights) using nothing more than a debounce timer and a click-timing window in software. No extra hardware required.

Controller 2 — Dual Joystick (with screen)

2

ANALOG STICKS

1

TFT DISPLAY



BIDIRECTIONAL

This controller adds a second analog stick and a small TFT display, which changes the protocol from one-way to bidirectional — the robot now broadcasts status back so the screen can show live telemetry.

Behavior

- ▶ Left / right stick → extended drive packet (throttle/steering or dual-axis, depending on mode)
- ▶ Same click-pattern speed/lights control as the single-stick controller
- ▶ New: receives the robot's heartbeat packet (~every 500ms) — signal strength, relay state, uptime

The HUD

The TFT renders the heartbeat as a HUD: relay/light status, an RSSI signal bar, a link-alive indicator, and uptime. The robot doesn't need to know which controller is connected — it always sends the heartbeat, and only a listening controller does anything with it.